Java Backend Architecture

Interview Series

Chapter 8.2 Authentication: Sessions, Cookies & JWT

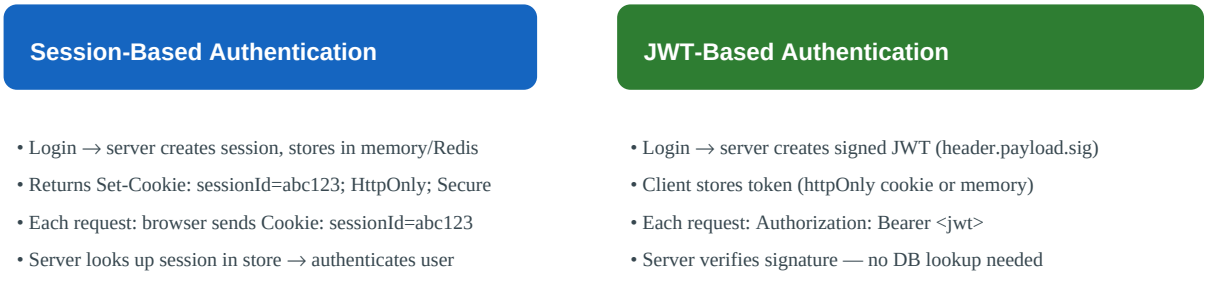
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 8.2 — Authentication: Sessions, Cookies & JWT

Introduction

Authentication verifies who a user is. The two dominant stateless/stateful models in web applications are **session-based authentication** (server stores state, client holds session ID in a cookie) and **JWT-based authentication** (server is stateless, client holds a self-contained signed token). Understanding the security trade-offs of each approach, cookie security attributes, and JWT vulnerabilities is essential for building secure backend systems.

Session vs JWT Authentication



Session: stateful (server must store state). JWT: stateless (self-contained, no server state needed).

Aspect	Session-Based	JWT-Based
State	Stateful — server stores session	Stateless — token is self-contained
Storage	Server memory / Redis / DB	Client-side (cookie or memory)
Revocation	Easy — delete session from store	Hard — token valid until expiry
Scalability	Requires shared session store	Horizontal scaling trivial
Payload size	Small (just session ID)	Larger (contains claims)
Performance	DB/cache lookup per request	No lookup — verify signature only
Suitable for	Monoliths, same-domain web apps	APIs, microservices, SPAs

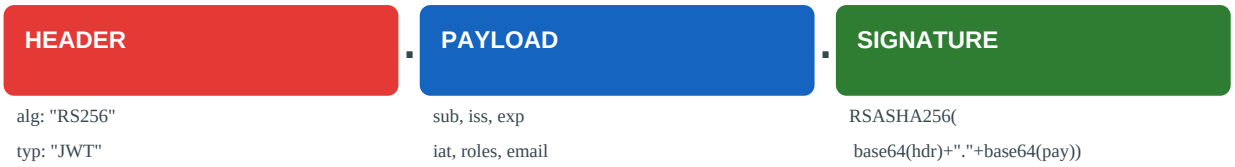
Cookie Security Attributes

Attribute	Value	Effect
HttpOnly	HttpOnly	JavaScript cannot read cookie — prevents XSS token theft
Secure	Secure	Cookie only sent over HTTPS — prevents plaintext transmission
SameSite	Strict	Cookie never sent cross-site — strongest CSRF protection
SameSite	Lax	Cookie sent on top-level navigation — good balance (default since Chrome 80)
SameSite	None; Secure	Cookie sent cross-site — required for OAuth, payment flows
Domain	example.com	Cookie shared with all subdomains of example.com

Path	/api	Cookie only sent to /api/* paths
Max-Age	3600	Cookie expires in 1 hour. Use instead of Expires.

```
// Spring Boot — secure cookie configuration @Configuration public class SessionConfig
implements WebMvcConfigurer { @Bean public CookieSerializer cookieSerializer() {
DefaultCookieSerializer serializer = new DefaultCookieSerializer();
serializer.setCookieName("SESSIONID"); serializer.setUseHttpOnlyCookie(true);
serializer.setUseSecureCookie(true); serializer.setSameSite("Strict");
serializer.setCookieMaxAge(3600); return serializer; } }
```

JWT Structure



All three parts are Base64URL-encoded. Only SIGNATURE provides integrity — PAYLOAD is readable by anyone.

Claim	Name	Description
iss	Issuer	Who issued the token (e.g., https://auth.example.com)
sub	Subject	User identifier (userId or email)
aud	Audience	Intended recipient (e.g., https://api.example.com)
exp	Expiration	Unix timestamp — token invalid after this time
iat	Issued At	Unix timestamp — when token was issued
nbf	Not Before	Token not valid before this timestamp
jti	JWT ID	Unique identifier — used to prevent replay attacks

JWT Implementation (Spring Boot)

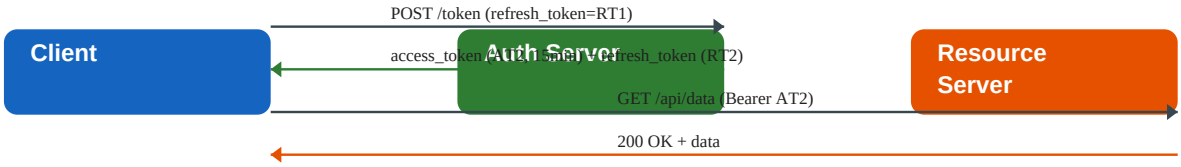
```
// JWT utility class using io.jsonwebtoken (jjwt) @Component public class JwtTokenProvider {
@Value("${jwt.secret}") private String secret; @Value("${jwt.expiration:900}") private long
expirationSeconds; private SecretKey key() { return
Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret)); } public String createToken(Authentication
auth) { UserDetails user = (UserDetails) auth.getPrincipal(); Instant now = Instant.now();
return Jwts.builder() .subject(user.getUsername()) .issuedAt(Date.from(now))
.expiration(Date.from(now.plusSeconds(expirationSeconds))) .claim("roles",
user.getAuthorities().stream() .map(GrantedAuthority::getAuthority).toList()) .signWith(key())
.compact(); } public Claims validateToken(String token) { return Jwts.parser()
.verifyWith(key()) .build() .parseSignedClaims(token) .getPayload(); } }
```

JWT Vulnerabilities

Vulnerability	Description	Fix
alg:none attack	JWT header sets alg to "none" — no signature verified	Explicitly whitelist allowed algorithms
RS256 → HS256	Server uses RS256 public key as HMAC secret	Whitelist algorithm per key type
Weak HMAC secret	Short or guessable secret allows brute-force	Use 256-bit+ random secret
No expiry	Token valid indefinitely	Always set exp claim (15-60 min for access tokens)

No audience check	Token from one service accepted by another	Validate aud claim matches expected audience
localStorage XSS	Script can steal token from localStorage	Store in httpOnly cookie or memory only

Refresh Token Rotation



RT1 is invalidated after use — RT2 issued. If RT1 is replayed, both tokens are revoked (theft detection).

Access tokens should be short-lived (5-15 minutes). Refresh tokens are long-lived (days-weeks) but stored securely. **Refresh token rotation**: every time a refresh token is used, a new refresh token is issued and the old one is invalidated. If an old refresh token is presented (replay attack), both tokens are revoked — indicating theft.

```
// Spring Security — JWT filter @Component public class JwtAuthFilter extends
OncePerRequestFilter { private final JwtTokenProvider jwtProvider; @Override protected void
doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) throws
ServletException, IOException { String token = extractToken(req); if (token != null) { try {
Claims claims = jwtProvider.validateToken(token); List<String> roles = claims.get("roles",
List.class); List<GrantedAuthority> authorities = roles.stream()
.map(SimpleGrantedAuthority::new).toList(); UsernamePasswordAuthenticationToken auth = new
UsernamePasswordAuthenticationToken( claims.getSubject(), null, authorities);
SecurityContextHolder.getContext().setAuthentication(auth); } catch (JwtException e) {
res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid token"); return; } }
chain.doFilter(req, res); } private String extractToken(HttpServletRequest req) { String header
= req.getHeader("Authorization"); if (header != null && header.startsWith("Bearer ")) { return
header.substring(7); } // Also check httpOnly cookie if (req.getCookies() != null) { for
(Cookie c : req.getCookies()) { if ("access_token".equals(c.getName())) return c.getValue(); }
} return null; } }
```

50+ Interview Questions & Answers

Q1. What is the difference between authentication and authorisation?

Authentication: verifying who you are (login, identity). Authorisation: determining what you're allowed to do (permissions, access control). Authentication always comes first.

Q2. What is session-based authentication?

Server creates a session after login, stores it (memory/Redis/DB), and sends the session ID in a Set-Cookie header. Client sends the cookie on every request; server looks up the session to authenticate.

Q3. What are the security attributes of cookies?

HttpOnly (no JS access), Secure (HTTPS only), SameSite (Strict/Lax/None for CSRF), Domain, Path, Max-Age/Expires. Always use HttpOnly + Secure + SameSite for session cookies.

Q4. What does SameSite=Strict mean?

Cookie is never sent on cross-site requests — not even on top-level navigation (clicking a link from another site). Strongest CSRF protection but breaks OAuth flows and cross-site linking.

Q5. What does SameSite=Lax mean?

Cookie is sent on top-level navigation (clicking links, GET form submissions) but not on cross-origin sub-requests (images, iframes, AJAX). Default since Chrome 80. Good balance for most apps.

Q6. What is JWT?

JSON Web Token — a compact, self-contained, signed token encoding JSON claims. Format: header.payload.signature (all Base64URL-encoded). The signature verifies authenticity and integrity.

Q7. What is the difference between HMAC-SHA256 (HS256) and RSA-SHA256 (RS256) in JWT?

HS256: symmetric — same secret key for signing and verification. All parties must share the secret. RS256: asymmetric — private key signs, public key verifies. Services can verify tokens without access to the signing key.

Q8. When would you use RS256 over HS256 for JWTs?

RS256 when multiple services need to verify tokens but shouldn't have signing ability (e.g., resource servers). HS256 when only one server signs and verifies. RS256 is standard for OAuth2/OIDC.

Q9. What are standard JWT claims?

iss (issuer), sub (subject/user ID), aud (audience), exp (expiration Unix timestamp), iat (issued at), nbf (not before), jti (JWT ID for uniqueness/replay prevention).

Q10. How do you revoke a JWT?

JWTs can't be invalidated before expiry without state. Options: (1) Short expiry (15 min) + refresh tokens, (2) Token blacklist/denylist in Redis (check jti on every request), (3) Secret rotation (invalidates all tokens).

Q11. What is the alg:none JWT attack?

Attacker modifies the JWT header to set alg='none' and removes the signature. A vulnerable library accepts unsigned tokens. Fix: explicitly whitelist allowed algorithms and reject 'none'.

Q12. What is the RS256 to HS256 attack?

If the server uses the RS256 public key as the HMAC secret for HS256 verification, an attacker can get the public key (it's public), sign a token with HS256, and the server accepts it. Fix: whitelist algorithm per token type.

Q13. Why is storing JWT in localStorage insecure?

JavaScript can read localStorage — XSS attacks can steal the token. HttpOnly cookies prevent JS access. Store short-lived tokens in memory (JS variable) and refresh tokens in httpOnly cookies for best security.

Q14. What is CSRF and how does it relate to cookies?

Cross-Site Request Forgery — a malicious site tricks the browser into sending a cookie-authenticated request. SameSite=Strict/Lax cookie attribute prevents this. CSRF tokens (double-submit or synchronizer pattern) provide additional protection.

Q15. What is refresh token rotation?

Each time a refresh token is used, a new refresh token is issued and the old one invalidated. If an old refresh token is presented again, it indicates theft and both tokens are revoked.

Q16. What is the recommended JWT expiry for access tokens?

5-15 minutes for access tokens (short-lived, reduces window for token misuse). Refresh tokens: hours-days for web, longer for mobile. Balance security (short) vs user experience (avoid constant re-login).

Q17. How do you securely transmit JWT to the client?

Option 1 (recommended): httpOnly + Secure + SameSite=Strict cookie for the access token. Option 2: In-memory JavaScript variable (lost on page refresh, needs refresh token in httpOnly cookie). Never in localStorage for sensitive apps.

Q18. What is a session fixation attack?

Attacker sets the session ID before authentication (e.g., via URL parameter). When the victim logs in using that session, the attacker already knows the session ID. Fix: always regenerate session ID after successful login.

Q19. What is session hijacking?

Stealing a valid session ID (via XSS, network sniffing, or log exposure) and using it to impersonate the user. Prevented by HttpOnly+Secure cookies, HTTPS, short session timeouts, and IP binding.

Q20. What is the difference between stateful and stateless authentication?

Stateful: server stores session data — easy revocation, requires distributed store for horizontal scaling. Stateless: JWT — no server state, scales trivially, but revocation requires additional infrastructure.

Q21. What is a PKCE code challenge in OAuth?

Proof Key for Code Exchange — prevents authorisation code interception. Client generates code_verifier (random), computes code_challenge=SHA256(code_verifier), sends challenge to authorisation server. When exchanging code for token, sends verifier — server validates.

Q22. What is the Bearer token scheme?

HTTP authentication scheme: Authorization: Bearer . The bearer (whoever has the token) can use it. Requires HTTPS to prevent token interception. Used for OAuth2 access tokens and JWT.

Q23. How does Spring Security process JWT tokens?

Implement OncePerRequestFilter to extract and validate JWT. On success, create UsernamePasswordAuthenticationToken and set in SecurityContextHolder. Configure SecurityFilterChain to use the filter and disable session creation.

Q24. What is a JWK Set (JWKS)?

JSON Web Key Set — a public endpoint (/.well-known/jwks.json) exposing public keys used to verify JWTs. Resource servers fetch the JWKS to validate tokens issued by the auth server without needing the private key.

Q25. What is token introspection?

OAuth2 mechanism (RFC 7662) where a resource server calls the auth server's /introspect endpoint with a token to verify its validity and retrieve claims. Useful for opaque (non-JWT) tokens or when fresh validation is required.

Q26. How do you handle token expiry in a SPA?

Intercept 401 responses in an Axios/Fetch interceptor. If access token is expired, silently call /refresh endpoint with the refresh token (from httpOnly cookie). Retry the original request with the new access token.

Q27. What is the difference between ID token and access token in OIDC?

ID token: JWT for the client application to know who the user is (identity claims). Access token: credential for accessing resource servers (permissions/scopes). ID tokens should NOT be sent to APIs.

Q28. What is a nonce in JWT/OIDC?

A random string included in the authentication request and embedded in the ID token. The client verifies the nonce matches — prevents replay attacks where an old ID token is reused.

Q29. What is multi-factor authentication (MFA)?

Authentication requiring two or more factors: something you know (password), something you have (TOTP code, hardware key), something you are (biometrics). TOTP (RFC 6238) is most common — 6-digit time-based code from apps like Google Authenticator.

Q30. How does TOTP work?

Time-based One-Time Password (RFC 6238): HMAC-SHA1(secret, floor(currentTime/30)). Both client and server compute the same code using the shared secret and 30-second time window. Codes expire every 30 seconds.